

JAVA PROGRAMMING

Chapter 13: Object-Oriented Programming in Java

Personal Notes

Object-Oriented Programming (OOP) is the backbone of modern software development, and Java is one of the languages that fully embraces this paradigm. By organizing code into objects and classes, OOP lets us build scalable, maintainable, and reusable applications.

If procedural code is like writing a recipe, OOP is like designing a kitchen. You don't just describe steps — you model real things (oven, fridge, chef) with their own properties and behaviors, and let them interact. That mental shift is what makes OOP so powerful.

This chapter walks through the core ideas of OOP in Java with clear examples for each.

1. Classes and Objects

At the heart of OOP lie two related ideas: classes and objects.

1.1 What is a Class?

A class is a blueprint — a template used to create objects. A class defines two things:

- State — variables / fields (what the object knows)
- Behavior — methods / functions (what the object can do)

Think of a class as the design for a real-world entity. The blueprint of a car defines that every car has wheels, color, and speed, and can drive, brake, and honk. The blueprint itself is not a car — it's just the design.

1.2 What is an Object?

An object is an instance of a class. When an object is created, memory is allocated for it, and it can use the properties and behaviors defined by the class.

If Car is the blueprint, then myRedHonda and yourBlueToyota are individual cars — actual objects built from that blueprint.

Example: A Student Class

```
public class Student {  
    // State (fields)  
    String name;  
    int age;  
    int marks;  
  
    // Behavior (method)  
    public void displayInfo() {
```

```

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Marks: " + marks);
    }
}

```

Now let's create some Student objects in main:

```

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Aman";
        s1.age = 18;
        s1.marks = 92;
        s1.displayInfo();

        Student s2 = new Student();
        s2.name = "Riya";
        s2.age = 20;
        s2.marks = 88;
        s2.displayInfo();
    }
}

```

Class vs Object — Visual

Class: Student		Objects (instances of Student)
<pre> +-----+ name age marks displayInfo() +-----+ </pre>	--new-->	<pre> s1: { name="Aman", age=18, marks=92 } s2: { name="Riya", age=20, marks=88 } </pre>
(blueprint – no memory)		(actual cars built from the blueprint)

Key insight: A class doesn't occupy memory until an object is created. Multiple objects can be created from a single class, and each one stores its own values.

2. Constructors

Setting fields manually one by one (`s1.name = ...`, `s1.age = ...`) gets tedious. Java gives you a cleaner way: constructors.

A constructor is a special method that runs automatically when you create an object using `new`. Its job is to set up the initial state of the object.

Rules

- Constructor name **MUST** match the class name exactly

- It has NO return type — not even void
- It is called automatically by new

Example

```
public class Student {
    String name;
    int age;

    // Constructor
    public Student(String n, int a) {
        name = n;
        age = a;
    }

    public void displayInfo() {
        System.out.println(name + ", " + age);
    }
}

// Usage:
Student s1 = new Student("Aman", 18);
s1.displayInfo();    // Aman, 18
```

If you don't write any constructor, Java gives you a default empty one automatically. The moment you write your own, the default disappears.

3. Encapsulation

Encapsulation is the idea of binding data (variables) and methods that work on that data together inside a single unit (a class) — while restricting direct access from outside.

Think of it like a capsule of medicine — all the ingredients are sealed inside, and you only interact with it through a controlled opening.

3.1 Why Encapsulation Matters

- Protects data from accidental or invalid changes
- Improves security by hiding internal details
- Makes code easier to maintain — you can change internals without breaking other code
- Lets you enforce rules (e.g., age cannot be negative)

3.2 How to Achieve It

Using Java's access modifiers:

Modifier	Where it's accessible
private	Accessible only within the same class
default	Accessible within the same package (no keyword)
protected	Accessible within the package AND in subclasses
public	Accessible from anywhere

3.3 Example with Getters and Setters

```
public class BankAccount {
    private double balance;    // hidden from outside

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}

// Usage:
BankAccount acc = new BankAccount();
acc.deposit(1000);
acc.withdraw(300);
System.out.println(acc.getBalance());    // 700

// acc.balance = -99999;    // ERROR – balance is private
```

Key insight: Instead of exposing variables directly, you wrap them with getters (read access) and setters (write access). This gives you full control — you can validate, log, or restrict any access.

4. Inheritance

Inheritance allows one class to inherit fields and methods from another class. This promotes code reuse — you write common features once in a parent class, and child classes get them for free.

4.1 Terminology

- Parent / Base / Superclass — the class being inherited FROM
- Child / Derived / Subclass — the class doing the inheriting
- extends — the Java keyword used to inherit

4.2 Example

```
// Parent class
public class Animal {
    String name;

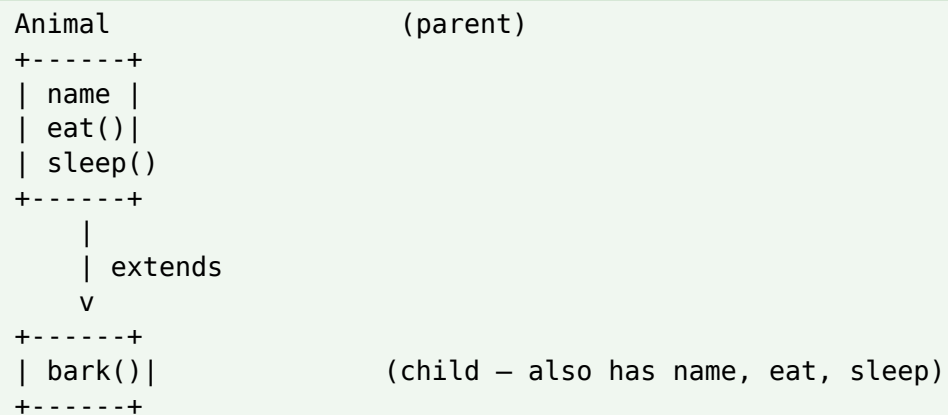
    public void eat() {
        System.out.println(name + " is eating.");
    }

    public void sleep() {
        System.out.println(name + " is sleeping.");
    }
}

// Child class
public class Dog extends Animal {
    public void bark() {
        System.out.println(name + " says: Woof!");
    }
}

// Usage:
Dog d = new Dog();
d.name = "Bruno";
d.eat();    // inherited from Animal
d.sleep();  // inherited from Animal
d.bark();   // defined in Dog
```

Inheritance Tree — Visual



4.3 Benefits

- Code reusability — write common logic once, reuse in many classes
- Reduced redundancy — no copy-pasting fields/methods
- Establishes IS-A relationships — a Dog IS-AN Animal

Important rule: Java supports SINGLE inheritance for classes — a class can extend only ONE parent. To get multiple-inheritance-like behavior, Java uses interfaces (covered in Section 6).

5. Polymorphism

Polymorphism literally means "many forms." In OOP, it means the same method behaving differently depending on context — different parameters, or different object types.

5.1 Compile-Time Polymorphism (Method Overloading)

Same method name, different parameter lists. The compiler decides which one to call based on the arguments.

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }

    public double add(double a, double b) {
        return a + b;
    }
}

// Usage:
Calculator c = new Calculator();
c.add(2, 3);           // calls first one → 5
c.add(1, 2, 3);        // calls second one → 6
c.add(2.5, 3.5);       // calls third one → 6.0
```

5.2 Run-Time Polymorphism (Method Overriding)

A child class provides its OWN version of a method that already exists in the parent class. Which version actually runs is decided at runtime based on the real object type.

```
public class Animal {
```

```

    public void sound() {
        System.out.println("Some generic sound");
    }
}

public class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("Woof!");
    }
}

public class Cat extends Animal {
    @Override
    public void sound() {
        System.out.println("Meow!");
    }
}

// Usage – the magic of polymorphism:
Animal a;
a = new Dog();    a.sound();    // Woof!
a = new Cat();    a.sound();    // Meow!

```

Why this is powerful: The variable 'a' is declared as Animal, but the actual method that runs depends on what type of object it actually holds at that moment. This is what lets you write flexible code that works with any subclass.

5.3 Overloading vs Overriding — Quick Comparison

Aspect	Overloading	Overriding
Where	Within the SAME class	Across PARENT and CHILD classes
Signature	Different parameters	Exact same signature
When decided	At compile time	At runtime
Also called	Compile-time polymorphism	Run-time polymorphism
Example	add(int,int) + add(int,int,int)	Dog overrides Animal's sound()

6. Abstraction

Abstraction means hiding the complex inner workings of something and only exposing the essential parts. You drive a car using a steering wheel and pedals — you don't care how the engine pistons fire. That's abstraction.

6.1 Why Abstraction?

- Reduces complexity — users see only what they need
- Improves readability — code reads at a higher level
- Increases security — internal details stay hidden
- Makes code easier to change — implementation can change without breaking users

6.2 How to Achieve Abstraction in Java

Java provides two tools:

- Abstract classes — partial abstraction (can have both abstract and regular methods)
- Interfaces — pure abstraction (originally only method signatures)

Abstract Class Example

```
public abstract class Shape {
    public abstract double area();    // no body – subclass MUST provide it

    public void describe() {          // regular method – inherited as-is
        System.out.println("This is a shape.");
    }
}

public class Circle extends Shape {
    double radius;

    public Circle(double r) {
        radius = r;
    }

    @Override
    public double area() {
        return 3.14 * radius * radius;
    }
}

// Usage:
Shape s = new Circle(5);
System.out.println(s.area());        // 78.5
s.describe();                        // This is a shape.

// Shape s2 = new Shape();    // ERROR – can't instantiate abstract class
```

Interface Example

```
public interface Drawable {
    void draw();    // implicitly public and abstract
}
```



```
public class Square implements Drawable {
    @Override
    public void draw() {
        System.out.println("Drawing a square");
    }
}

public class Circle implements Drawable {
    @Override
    public void draw() {
        System.out.println("Drawing a circle");
    }
}
```

Abstract class vs Interface: Use an abstract class when classes share some COMMON code (a partial implementation). Use an interface when you only care about WHAT the class can do, not how. A class can extend ONE abstract class but implement MANY interfaces — that's how Java achieves multiple inheritance safely.

7. Modularity

Modularity is the practice of breaking a large program into small, independent, manageable pieces. Each piece does one well-defined job and can be developed, tested, and updated separately.

Advantages

- Easier debugging — bugs are isolated to one module
- Better teamwork — different people can work on different modules
- Improved code organization
- Easier to test and reuse modules across projects

How Java Achieves Modularity

- Classes — bundle related fields and methods together
- Packages — group related classes (like folders for code)
- Methods — break logic into reusable named blocks
- Modules (Java 9+) — group packages into named modules with explicit dependencies

8. The Four Pillars of OOP

All of OOP can be summarized into four foundational pillars. Memorize these — they come up in nearly every Java interview.

Pillar	What it means
Encapsulation	Bundling data + methods inside a class and hiding internals via private + getters/setters
Inheritance	One class inheriting fields and methods from another class using extends
Polymorphism	Same method name doing different things based on parameters (overloading) or object type (overriding)
Abstraction	Hiding implementation details and exposing only essential behavior using abstract classes or interfaces

9. Real-World Analogy: Car Manufacturing

OOP becomes much more intuitive when mapped to a real-world example. Imagine a car factory:

The Mapping

- Each component (engine, wheels, doors) is designed separately → CLASSES
- Each car produced from those designs is an INSTANCE → OBJECT
- Common features reused across models (Sedan, SUV, Hatchback all share Vehicle features) → INHERITANCE
- Same function behaves differently across cars (manual brakes vs anti-lock brakes) → POLYMORPHISM
- Internal mechanisms hidden from the driver — you press a pedal, the car does the rest → ABSTRACTION
- Internal components stay private inside the engine — you can't poke at the pistons → ENCAPSULATION
- Factory divided into engine plant, assembly plant, paint shop → MODULARITY

Every OOP concept you learn maps to something this concrete. When in doubt, ask: "How would this work in the car factory?"

10. Common Beginner Mistakes

Mistake 1: Confusing class with object

A class is the blueprint. An object is the actual thing built from it. Student is a class; new Student() creates an object.

Mistake 2: Making everything public

Don't expose every field as public. Use private for fields and provide controlled access via getters/setters.

Mistake 3: Forgetting @Override

When overriding a parent method, add @Override above it. It's not required, but it lets the compiler catch typos — if you misspell the method name, @Override will throw an error instead of silently creating a new method.

Mistake 4: Trying to instantiate an abstract class

```
Shape s = new Shape();    // ERROR if Shape is abstract
Shape s = new Circle(5); // OK – create a concrete subclass
```

Mistake 5: Confusing extends and implements

- extends — used for class inheritance (only ONE parent class)
- implements — used to implement interfaces (can implement MANY)

Mistake 6: Overloading by return type only

Two methods cannot differ ONLY in return type. The parameter list must differ.

```
int add(int a, int b)    { ... }
double add(int a, int b) { ... } // ERROR – same parameters
```

11. Quick Reference Table

Keyword	Meaning
class	Defines a blueprint for objects
new	Creates an actual object from a class
this	Refers to the current object inside a method
private	Access only within the same class
public	Access from anywhere
protected	Access in same package + subclasses
extends	Inherit from a class
implements	Implement an interface
super	Refers to the parent class (call parent constructor or method)
abstract	Marks a class or method as incomplete (must be subclassed)
interface	Pure abstraction — defines what, not how
@Override	Annotation marking a method as overriding a parent method
final	Prevents inheritance, overriding, or reassignment

Keyword	Meaning
<code>static</code>	Belongs to the class itself, not to any object

12. Why OOP Matters in Java

OOP isn't just an academic topic — it shapes how real Java code is written every day. Mastering it lets you:

- Build scalable applications that can grow without becoming a mess
- Reuse code across projects through inheritance and interfaces
- Maintain code easily — fix bugs in one place without breaking everything
- Write clean, structured code that other developers can read
- Crack interviews — OOP questions are a guaranteed part of every Java interview

13. Practice Problems

Practice these to lock in the concepts. Try to implement each one without looking back at the chapter.

1. Create a `Book` class with `title`, `author`, `price`. Make all fields private and add getters/setters.
2. Build a `Vehicle` class with two subclasses `Car` and `Bike`. Add a `start()` method that prints differently for each.
3. Create an abstract `Shape` class with `area()`. Provide concrete subclasses `Circle`, `Rectangle`, and `Triangle`.
4. Build a `Calculator` class with overloaded `add()` methods for `int`, `double`, and three-argument `int`.
5. Design a `BankAccount` class with `deposit`, `withdraw`, and `balance check`. Use proper encapsulation.
6. Create an interface `Playable` with method `play()`. Implement it in `Guitar`, `Piano`, and `Drums` classes.
7. Build an `Employee` class. Create a `Manager` subclass that overrides a `calculateSalary()` method.
8. Design a `Library` system with classes `Book`, `Member`, and `Library`, with appropriate methods and fields.

14. Final Takeaway

OOP is not just "another Java topic" — it is the way Java thinks. Every real-world project, every framework you'll ever use (Spring, Hibernate, Android), is built on top of these four pillars: Encapsulation, Inheritance, Polymorphism, and Abstraction.

Get comfortable here, and the rest of your Java journey — design patterns, frameworks, system design — becomes much easier. OOP is the lens through which every serious Java developer sees code.

Key Takeaway: Stop thinking in terms of "variables and functions." Start thinking in terms of "objects with state and behavior, interacting with each other." Once that mental switch happens, OOP stops being a topic to study and becomes the natural way you write code.

15. What's Next?

Once OOP feels natural, the natural next steps are:

- Exception handling — try, catch, finally, throw, throws
- Collections — ArrayList, HashMap, HashSet, and more
- Generics — writing code that works with any type
- Design Patterns — Singleton, Factory, Observer, Strategy
- SOLID principles — guidelines for clean OOP design
- Multithreading and concurrency
- Frameworks like Spring Boot for real-world apps